**Understanding Your Data (Day 19)**

**Course:** 100 Days of Machine Learning

**Teacher:** CampusX (Nitish Singh)

**Introduction**

When you receive a new dataset for a Machine Learning project, the very first step is not building a model. The first step is to **understand the data**.

This is the first video in a 4-part series focused on data exploration:

1. **Day 19 (Today):** Asking basic questions to understand the data structure.
2. **Day 20:** Univariate Analysis (Exploratory Data Analysis - EDA).
3. **Day 21:** Multivariate Analysis (EDA).
4. **Day 22:** Automated EDA using Pandas Profiling.

**The Dataset**

For this explanation, we are using the famous **Titanic Dataset** from Kaggle. This is a standard dataset for beginners in Machine Learning.

**Initial Setup (Code)**

```
import pandas as pd


# Loading the dataset

df = pd.read_csv('train.csv')
```

**Line-by-line Explanation:**

1. import pandas as pd: We import the Pandas library, which is the standard tool for data manipulation in Python.
2. df = pd.read_csv('train.csv'): We use the read_csv function to load the Titanic data into a DataFrame named df.

**7 Basic Questions to Ask Your Data**

**1. How big is the data?**

Before you start any analysis, you must know the size of the dataset you are dealing with (how many rows and columns).

**Code:**

```
df.shape
```

**Line-by-line Explanation:**

- df.shape: This attribute returns a tuple representing the dimensionality of the DataFrame. It shows (number of rows, number of columns).

**Why it matters:** Knowing the size helps you decide if you can process the data on your local machine or if you need a cloud-based solution for "Big Data."

**2. How does the data look?**

It is always good practice to see a few rows of the data to understand what kind of information is stored in each column.

**Code (Method A - Head):**

df.head()

- **Explanation:** Shows the first 5 rows of the dataset.

**Code (Method B - Sample):**

df.sample(5)

- **Explanation:** Shows 5 random rows from the dataset.

**Teacher's Tip (The "Bias" Problem):** Sometimes, data is sorted in a specific way (e.g., all children first, then adults). If you only use df.head(), you might get a wrong idea about the data. Using df.sample() is a better strategy because it gives you a random overview and helps avoid biased conclusions.

**3. What is the data type of the columns?**

Every column in your data has a specific type (Integer, Float, Object/String).

**Code:**

df.info()

**Line-by-line Explanation:**

- df.info(): This function provides a summary of the DataFrame, including the index range, column names, non-null counts, and data types.

**Important Reasoning (Memory Optimization):**

- The output shows how much memory the data occupies.
- Sometimes numerical columns are stored as float64 when they could be int32.
- **Trick:** By converting data types to more efficient ones, you can reduce memory usage and speed up your Machine Learning algorithms.

**4. Are there any missing values?**

Missing values (NaN) are a major problem in Machine Learning. You need to know which columns have empty cells.

**Code:**

df.isnull().sum()

**Line-by-line Explanation:**

1. df.isnull(): This creates a table of True/False values (True if data is missing).

2. .sum(): This adds up all the "True" values for each column, giving you the total count of missing values per column.

**Teacher's Reasoning:**

- In the Titanic data, the 'Cabin' column often has many missing values. If more than 50-70% of data is missing, you might decide to delete that column.
- For columns like 'Age', you might fill the missing values with the mean or median.

**5. How does the data look mathematically?**

You need to see the "High-Level Mathematical Summary" of the numerical columns.

**Code:**

df.describe()

**Line-by-line Explanation:**

- df.describe(): This generates statistics for numerical columns:
    - **count**: Number of non-empty values.
    - **mean**: The average value.
    - **std**: Standard Deviation (how spread out the data is).
    - **min**: The smallest value.
    - **25%**: The 25th percentile (1st Quartile).
    - **50%**: The median (2nd Quartile).
    - **75%**: The 75th percentile (3rd Quartile).
    - **max**: The largest value.

**Example Insight from Video:** By looking at the 'Age' column in describe(), we see the average age is ~30, but the youngest person was 0.42 years old and the oldest was 80. This helps you identify the age group of the passengers instantly.

**6. Are there duplicate values?**

Duplicate data can confuse your model and lead to overfitting (the model just memorizing patterns).

**Code:**

df.duplicated().sum()

**Line-by-line Explanation:**

1. df.duplicated(): Identifies rows that are exact copies of previous rows.
2. .sum(): Returns the total number of duplicate rows.

**Key Point:** If the output is 0, it means every row is unique. If you find duplicates, you should usually remove them using df.drop_duplicates().

**7. How is the correlation between columns?**

Correlation helps you understand how one column affects another.

**Code:**

df.corr()['Survived']

**Line-by-line Explanation:**

1. df.corr(): Calculates the **Pearson Correlation Coefficient** for all pairs of numerical columns. Values range from -1 to 1.
2. ['Survived']: We specifically look at how other columns relate to our "Target" (whether the person lived or died).

**Mathematical Reasoning:**

- **1.0 (Positive):** As column A increases, column B increases. (e.g., Higher Fare = Higher Survival chance).
- **-1.0 (Negative):** As column A increases, column B decreases. (e.g., Higher Pclass/lower class = Lower Survival chance).
- **0:** No relationship.

**Insight:** In the Titanic data, PassengerId has almost 0 correlation with Survived. This tells us that PassengerId is a useless column for prediction and we can delete it.

**Summary / Key Highlights**

- **Don't skip these steps:** These questions give you "Intuition" about your data.
- **Memory Matters:** Use .info() to check if you can optimize data types.
- **Correlation is Key:** Use .corr() to identify which features (columns) are actually important for your prediction.
- **Sample over Head:** Always use df.sample() to get a non-biased view of the data.

**Next Step:** In the next video, we will learn how to visualize these findings using graph